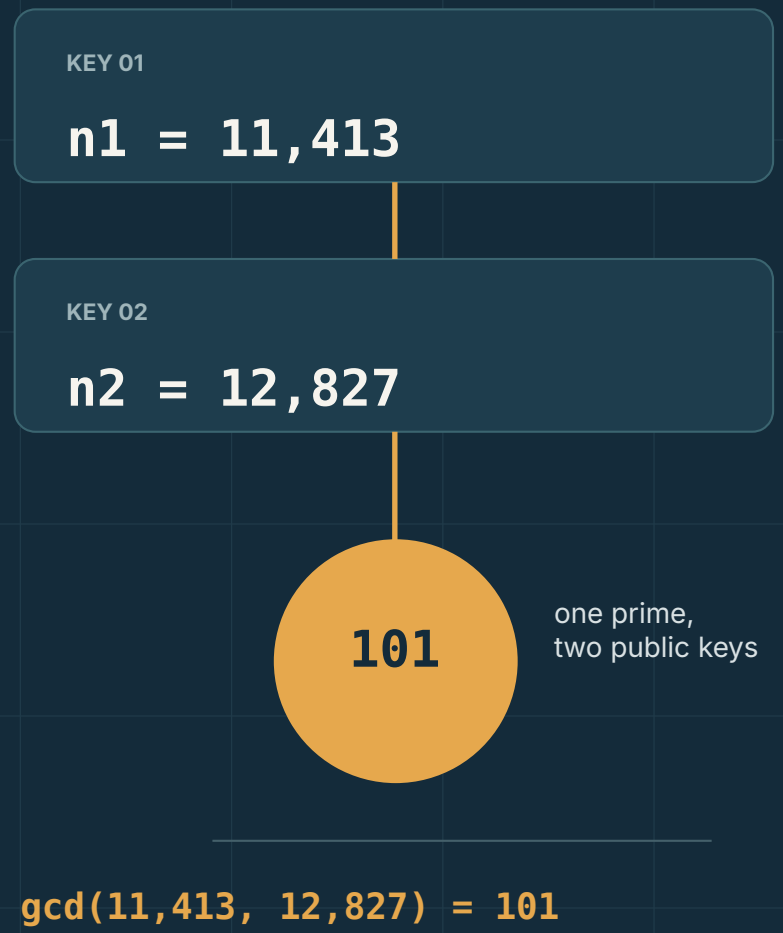
RSA keyring audit

Finding shared-prime failures with Batch-GCD

A code-first walkthrough of the mathematics, implementation, recorded tests, and limits.

github.com/unitin610-blip/delta-code_information-theory

THE FAILURE IN ONE PICTURE



RSA breaks when two keys reuse a prime.

The audit is looking for a key-generation failure, not a flaw in modular exponentiation.

KEY 01

11,413 = 101 x 113

KEY 02

12,827 = 101 x 127

101

THE GCD GIVES IT AWAY

gcd(n1, n2) = 101

Both moduli are now factorable;
the associated private keys can be derived.

One input file, one audit path, two outputs.

The Python script coordinates the audit; the Batch-GCD module does the shared-factor search.

01

Read CSV

Load each row and its modulus.

02

Validate

Reject empty input, missing n, or n <= 1.

03

Batch-GCD

Build product and remainder trees.

04

Classify

Separate factors, ambiguous cases, no finding.

05

Derive values

Compute p, q, phi(n), and private exponent d.

06

Write results

Update the CSV and generate a Markdown report.

OUTPUT ARTIFACTS

`compromised_keys (1).csv`

Factors, status, and demo fields

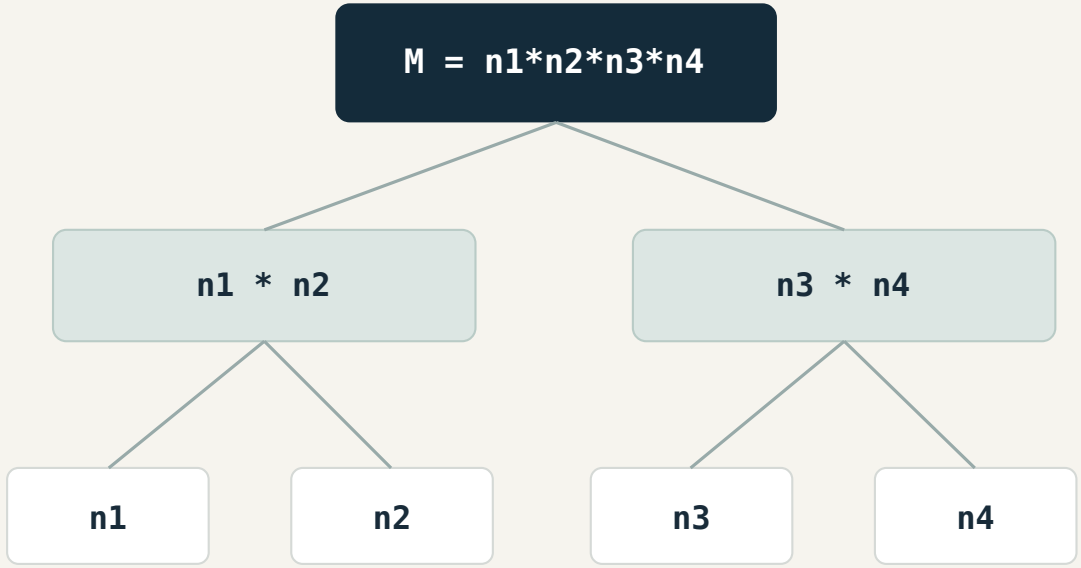
`remediation_report (1).md`

Method, recorded checks, limitations, mitigations

Batch-GCD turns a key list into a tree problem.

Rather than compute every pairwise GCD, reuse products across the full set of moduli.

PRODUCT TREE / EXAMPLE WITH FOUR MODULI



Odd-sized levels carry the final node upward unchanged.

REMAINDERS → CANDIDATE FACTORS

For each modulus n_i, the remainder tree gives:

$$r_i = M \bmod (n_i^2)$$

Then the shared-factor candidate is:

$$g_i = \gcd(n_i, r_i / n_i)$$

```
1 < g_i < n_i   ->  non-trivial factor
g_i = 1   ->  no shared factor detected
g_i = n_i   ->  ambiguous; inspect separately
```

The report cites O(N log^2 N) arithmetic operations under balanced-tree analysis; bit complexity still depends on integer size and multiplication.

Once p is known, the private exponent follows.

The checked-in script uses the factorization to demonstrate the RSA math on a toy key.

KEY 01 / DERIVATION

01 **n = 11,413**
The public modulus.

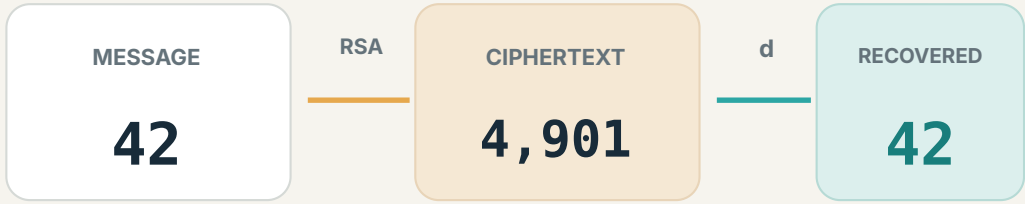
02 **p = 101, q = 113**
q = n / p

03 **phi(n) = 11,200**
(p - 1) * (q - 1)

04 **d = 7,873**
Inverse of 65,537 modulo phi(n).

SYNTHETIC ROUND-TRIP

The script encrypts a controlled message with (n, e).




```
CHECK        decrypted == original  ->  PASS
```

This proves the toy round-trip works. It is not decryption of an organizer-provided ciphertext.

The checked-in sample contains two compromised toy keys.

The CSV and Markdown report agree on the shared factor and the synthetic decryption demonstrations.

CURRENT CSV RESULT

KEY	MODULUS	SHARED P	STATUS
01	11,413	101	COMPROMISED
02	12,827	101	COMPROMISED

Demo plaintexts: 42 and 43. Both recorded RSA round-trips pass.

SEPARATE TESTS

2 / 2

sample moduli flagged with shared factor 101

2048-bit

synthetic PKCS#1 v1.5 payload test: PASS

SYNTHETIC BENCHMARK

10,000 small moduli

Saved notebook runs range from about 0.63 to 1.20 seconds. These are products of small primes, not a representative 2048-bit key population.

Keep the claims as narrow as the tests.

The repository contains useful proof-of-concept checks, but the recorded inputs are mostly synthetic.

WHAT IS DEMONSTRATED

Shared-prime detection

The toy moduli expose a common factor of 101.

Private-exponent derivation

Recovered factors are used to calculate d and test a controlled round-trip.

Separate padding check

A generated 2048-bit key passes a synthetic PKCS#1 v1.5 structure and payload test.

NOT ESTABLISHED BY THESE RESULTS

Challenge-ciphertext decryption

The repository report says no organizer-provided ciphertext was supplied; the messages are generated by the notebook/script.

Production-scale performance

The benchmark uses small synthetic moduli, so its timing does not predict full-size RSA workload performance.

IMPLEMENTATION NOTE

The duplicate-modulus fallback filters candidates with `other != n`, which also skips equal-valued duplicate rows. A direct `batch_gcd([])` call is not handled cleanly either; add regression tests for both edges.

Detection is the start of the response.

A shared factor is a compromise signal. The audit does not rotate keys or repair a key generator automatically.

01 **Contain**

Revoke and replace confirmed compromised keys. Never reuse the recovered private key.

02 **Audit the fleet**

Check related key inventories for shared factors before replacement keys go live.

03 **Fix key generation**

Use a vetted cryptographic library and a properly initialized CSPRNG with sufficient entropy.

04 **Keep evidence**

Restrict access to private material, rotate exposed credentials, and retain audit logs.

PREVENT RECURRENCE

Watch for repeated primes, weak entropy at boot, and cloned VM or device state.

Treat each finding as an incident to investigate, not merely a row to mark red in a report.

`AUDIT -> ROTATE -> VERIFY`

Two Python files power the audit.

The Batch-GCD module does the search; the audit script owns input, key derivation, test output, and reporting.

`batch_gcd.py`	Build product/remainder trees; return a GCD candidate for each modulus.
`audit_rsa (1).py`	Load and validate CSV; derive RSA values; run the demo; write the report and benchmark.
`Untitled12_RSA_Final_Review (1).ipynb`	Exploratory tests, output snapshots, synthetic padding test, and timing runs.
`compromised_keys (1).csv`	Two toy moduli with factors, derived private exponents, and synthetic round-trip fields.
`remediation_report (1).md`	Summary, algorithm, complexity note, test record, limitations, and mitigations.

SOURCE / github.com/unitin610-blip/delta-code_information-theory

THE MAIN TAKEAWAY

One reused prime can turn a public modulus into a factorization problem.

Batch-GCD makes the search practical across a key set. The conclusion must stay tied to the data actually tested.